

Cherry — DealFlow

Development Master Plan

Agile/Scrum execution model, technology strategy, development phases, quality, release and continuous improvement

Version 1.0 • September 2026 • Controlled Working Document

1. Purpose

This master plan defines how DealFlow will be prepared, built, tested, released, operated and continuously improved before detailed Jira sprint planning.

The later Jira document will translate this plan into epics, stories, tasks, acceptance criteria, dependencies and sprint commitments.

2. Development Objective

Build a production-quality, multi-tenant real-estate SaaS product while keeping the customer experience simple and the technical foundation extensible.

Primary objective: reach a commercially useful V1 quickly without creating architectural or quality problems that prevent continuous improvement.

3. Methodology

Agile development with Scrum.

Default sprint length: two weeks.

Core events: Sprint Planning, Daily Scrum, Backlog Refinement, Sprint Review and Retrospective.

Flow: discovery → prioritization → sprint → release → measurement → learning → next sprint.

4. Delivery Model

Vision → Roadmap → Epics → Features → User Stories → Tasks → Sprint → Release.

Prefer vertical slices that produce working capabilities across UI, API, domain, data and tests.

Definition of Ready: clear problem, value, acceptance criteria, dependencies and enough design information.

Definition of Done: implementation, tests, validation, security/permission checks, observability where relevant, documentation/migration work, review and release readiness.

5. Technology Strategy

Backend: Python, FastAPI, Pydantic, SQLAlchemy, Alembic.

Database: PostgreSQL.

Frontend: React, TypeScript, Vite.

Background/caching: Redis and a worker mechanism such as Celery when justified.

Files: S3-compatible object storage; MinIO locally.

Testing: pytest, Vitest, React Testing Library, Playwright.

Infrastructure: Docker/Docker Compose.

CI/CD: GitHub Actions.

API: REST initially, with event/webhook patterns where appropriate.

Rule: choose mature, simple technology that solves today's requirement; avoid infrastructure for hypothetical scale.

6. Architecture Strategy

Start as a modular monolith, not microservices.

Core domains: identity, tenancy, contacts, customers, leads, requirements, properties, activities, follow-ups, visits, deals, documents, imports, notifications, automation, billing, audit and analytics.

Future service extraction is allowed only when measured scale, reliability, ownership or integration requirements justify it.

Priorities: tenant isolation, clear boundaries, testability, observability, security, versioned contracts and safe migrations.

7. Repository & Engineering Structure

Dedicated DealFlow repository, separate from Onera.

Use main/develop and focused feature branches.

Initial areas: backend, frontend, infrastructure, docs and scripts.

Maintain formatting, linting, type checking, tests, commit standards, review practices and ADRs for material decisions.

8. Development Phases

Phase 0 — Preparation: product boundary, repository, environments, documentation, Scrum setup, CI and architecture/security baseline.

Phase 1 — Platform Foundation: FastAPI, React, PostgreSQL, Redis, Docker, configuration, migrations, health checks, logging, CI and observability.

Phase 2 — Identity & Tenant Foundation: organizations, tenants, users, authentication, roles, permissions, tenant isolation and audit.

Phase 3 — Core Business Data: contacts, customers, leads, requirements and properties.

Phase 4 — Data Import: CSV/Excel, mapping, validation, duplicates, preview, import batches, errors and historical/imported state.

Phase 5 — Lead Operations: pipeline, status, interest, outcome, ownership, activities, notes, timeline, search, filters and bulk actions.

Phase 6 — Follow-Up Intelligence: due/overdue/upcoming/waiting, reminders, recurring follow-ups, next actions and Today's Attention.

Phase 7 — Matching & Site Visits: matching, explanations, shortlist, scheduling, confirmation, reminders, feedback and next action.

Phase 8 — Deal Operations: negotiation, deal workspace, tasks, documents, approvals and payment/commission foundations.

Phase 9 — Automation: event/condition/action workflows, assignment, escalation and notifications.

Phase 10 — Billing & Cherry Control Plane: plans, subscriptions, entitlements, invoices, provider integration, webhooks, reconciliation, upgrades, downgrades, cancellation, grace period, suspension/restoration and internal administration.

Phase 11 — AI: extraction, summaries, matching assistance, next-action recommendations, drafting and insights under permission/policy controls.

Phase 12 — Production Scale & Continuous Evolution: advanced analytics, integrations, builder capabilities, channel partners, multi-branch and evidence-driven future capabilities.

9. V1 Target

Commercially useful flow: tenant → user → import/create contact → lead → requirement → property → match → follow-up → Today's Attention → pipeline.

Supporting V1: basic documents, timeline/history, audit foundation, search/filtering, data import and essential dashboards.

V1 is complete when the core customer workflow works reliably end-to-end, not when individual screens exist.

10. Sprint Model

Sprint 0 establishes the development baseline.

Each subsequent sprint has one clear goal and a small set of related stories.

New ideas go to the backlog unless they are critical, security-related, production-stability related or explicitly prioritized.

Sprint review demonstrates working software; retrospective produces process improvements.

11. Initial Sprint Sequence

Sprint 0 — Preparation.

Sprint 1 — Platform foundation.

Sprint 2 — Identity and tenancy.

Sprint 3 — Contacts/customers.

Sprint 4 — Leads.

Sprint 5 — Requirements/properties.

Sprint 6 — Data import.

Sprint 7 — Lead operations/timeline.

Sprint 8 — Follow-ups/Today's Attention.

Sprint 9 — Matching.

Sprint 10 — Site visits.

Sprint 11 — Deal workspace.

Sprint 12 — Documents.

Sprint 13 — Automation.

Sprint 14 — Billing/subscriptions.

Sprint 15 — Cherry Admin Control Plane.

Sprint 16 — Security hardening/production readiness.

Sprint 17+ — AI and continuous releases.

These are planning placeholders; the separate Jira plan will define exact stories and commitments.

12. Testing & Quality

Unit tests for business rules.

Integration tests for persistence and boundaries.

API tests for contracts, validation and authorization.

End-to-end tests for critical journeys.

Data tests for imports, duplicates, partial failures, retries and historical preservation.

Security tests for tenant isolation and authorization.

Production defects should become regression tests where appropriate.

13. Security Model

Security is part of every relevant feature, not a final phase.

Consider authentication, authorization, tenant isolation, sensitive data, auditability, input validation, secrets, dependencies and failure modes.

AI receives only authorized context and cannot bypass application permissions. High-risk actions require policy evaluation and approval where appropriate.

Backups, recovery, monitoring, incident response and security logging must exist before the product becomes business-critical.

14. Data Evolution

Customer data is a first-class product asset.

Schema changes are explicit and tested.

Migrations are controlled.

New capabilities should coexist with older records where practical.

No silent destructive migrations.

Import and audit history must remain understandable.

API changes use deliberate compatibility and deprecation practices.

15. Release Strategy

Flow: feature branch → tests → review → merge → CI → staging → validation → production.

Patch releases: fixes/security. Minor releases: backward-compatible capabilities. Major releases: significant changes.

Use feature flags or staged rollout when risk justifies them.

Release checklist includes scope, tests, migrations, security, observability, deployment, rollback/recovery, documentation and release notes.

16. Observability & Operations

Monitor API errors, latency, database health, background jobs, queues, storage, notifications, integrations and AI failures.

Monitor active tenants, users, leads, follow-ups, visits, deals, imports and billing events.

Customer health should include subscription, usage, errors, import state, storage, support and important security events.

Cherry will operate a separate internal control plane for tenants, billing, support, system health, incidents and audit.

17. Billing Development

Use a payment provider for payment rails rather than implementing payment collection ourselves.

Maintain Cherry-side records for plans, subscriptions, entitlements, invoices, payments, refunds/credits, provider references and webhook events.

Lifecycle: signup → tenant → plan → checkout → payment/mandate → verified webhook → activation → entitlements → renewal → recovery/cancellation.

Use a provider-adaptor boundary so the application is not permanently coupled to one provider.

18. Continuous Improvement

V1 is a launch milestone, not product completion.

Loop: customer problem → evidence → backlog → design → build → test → release → measure → learn → improve.

Inputs include customer feedback, support, analytics, incidents, sales objections, operational cost and strategy.

Every release should improve value, usability, reliability, security, performance, automation, observability or maintainability where relevant.

19. Documentation Lifecycle

Maintain controlled documentation for business requirements, product requirements, UX, domain/data model, architecture, security, APIs, testing, operations, roadmap, support, legal/compliance and ADRs.

Each topic should have one primary source of truth.

Material product or architecture changes update the relevant documents.

The Jira sprint document should reference these documents rather than duplicate them.

20. Ownership

Product owns vision, requirements, roadmap and prioritization.

Engineering owns implementation, architecture and technical quality.

Design owns UX and interaction quality.

QA owns quality strategy and release evidence.

Security/engineering owns security controls and reviews.

Operations owns deployment, monitoring, incidents and recovery.

Business/finance owns commercial and billing controls.

Company leadership owns strategic direction and final trade-offs during the initial solo phase.

21. Completion Strategy

Measure progress by completed customer workflows, not number of screens/endpoints/files.

Primary V1 test: can a real customer safely bring existing data into DealFlow, manage a lead, capture a requirement, find a property, complete follow-ups and understand what needs attention?

Operational readiness: can Cherry deploy, monitor, support, secure, back up and recover the system without manually changing code for each customer?

Commercial readiness: can a customer sign up, receive a tenant, use the product and eventually pay/renew through the billing system?

22. Fast Execution Rules

Do not over-design future features before their development phase.

Do not compromise tenant isolation or foundational security for speed.

Do not introduce microservices merely for appearance.

Do not create customer-specific forks where configuration can solve the need.

Do not let V1 scope expand indefinitely.

Do not sacrifice tests on critical workflows.

Prefer a small working increment over a large unfinished implementation.

23. Immediate Next Step

Create a separate DealFlow Jira Sprint & Execution Plan from this master document.

That document will contain epics, stories, tasks, acceptance criteria, dependencies, priorities, sprint goals, estimates and the exact first development sequence.

After that, begin Sprint 0/1 implementation rather than continuing broad ideation.